Edge Cases — Plain-Language Guide

What this document is

This explains the eight "edge cases" added to the Distributed Backup System after the D1 submission. An edge case is a situation that does not happen every day, but when it does happen it can lose or corrupt a backup. The list is written in plain language so it can be shown to a teacher or external reviewer without assuming networking or programming knowledge. A short "what changed technically" section at the end gives the concrete files and code locations to back it up.

The eight edge cases

1. The backup server dies in the middle of a copy

Everyday picture: You are photocopying a long document and the copier jams halfway through. You do not want to start from page one when it is fixed.

What could go wrong: Half a file sits on the server. The next backup overwrites it, mixing old and new halves — or nothing is recoverable.

What our system does: The client remembers the last chunk that arrived safely (a "resume point") and reports it before giving up. The backup controller retries the same server. If the server is still down, it switches to the second backup server and starts fresh there, labeling the run "restarted on backup." The dashboard shows the retry so the user is never left guessing.

2. Your own computer crashes during the backup

Everyday picture: Your laptop battery dies while a file is uploading. The server has the first half; nobody knows which chunk was the last good one.

What could go wrong: A later attempt starts from zero and overwrites the partial copy, wasting time and disk space.

What our system does: The dashboard finds the partial file on the server and offers "Resume from chunk N" (N is worked out from the partial file's size) or "Start over." Resuming skips chunks the server already has.

3. Re-saving a file destroys the previous good copy

Everyday picture: You save a new version of your essay over yesterday's version, but the new save fails halfway. Now both versions are ruined.

What could go wrong: The server starts overwriting the stored file as soon as the new transfer begins, so a failed re-backup deletes the only good copy.

What our system does: The server writes the new file to a temporary name (`filename.part`) and only swaps it into place after the last chunk arrives and checks out. If the transfer fails, the previous file is untouched. Temporary `.part` files are hidden from the dashboard so nobody mistakes them for finished backups.

4. The backup disk is full or locked

Everyday picture: You try to save to a USB stick that is full — or write-protected. The copy silently stops partway and you think it finished.

What could go wrong: Bytes are sent to a server that cannot store them; the client sees a closed connection and the dashboard blames the network instead of the disk.

What our system does: The server checks disk space before accepting a file (quota reservation) and catches write errors, sending back a clear "storage error." The client fails fast with `[CLIENT] Server storage error` instead of hanging. The dashboard can auto-route to the other server when the chosen one is full, and refuses outright when both are full.

5. Two people back up the same file name at once

Everyday picture: Two students both name their file `assignment.pdf` and send it to the same server at the same time. Their bytes get mixed into one file.

What could go wrong: One stored file contains half of student A's work and half of student B's — corrupted for both.

What our system does: The server "claims" the file name when the first transfer starts. A second client sending the same name is rejected with a clear "file busy" message, and the first file is left byte-identical and complete. Source names ending in `.part` are reserved and rejected so they cannot collide with the server's temporary files.

6. The source file is edited while it's being copied

Everyday picture: You change a number in a spreadsheet while it is still being copied to the server. Every chunk sent is valid on its own, but the stored file is half old, half new — and every checksum passes.

What could go wrong: A corrupted stored file looks correct because each chunk was checked individually.

What our system does: Before copying, the client snapshots the file's size and modification time. After the last chunk, it re-checks. If either changed, it stops with `[INCONSISTENT] Source changed during backup` and fails the run instead of reporting success. Known limit: an edit that keeps both size and timestamp identical can slip past; we state that limit instead of claiming full detection.

7. The system says "done" but nothing was actually saved

Everyday picture: You get a confirmation that your file uploaded, but the folder on the server is empty — or the file landed on the wrong server and nobody checked.

What could go wrong: A false sense of security. You delete your only copy believing it is backed up.

What our system does: When a run reports finished, the dashboard checks both backup servers (`stored_files/h3` and `stored_files/h4`) for the file and its

expected size. It marks the run "verified" or flags "done but copy not found" and logs the check, so a missing file can never show as a clean success.

8. Neither backup server is reachable

Everyday picture: Both destination computers are switched off or unplugged.

What could go wrong: The client hangs for a minute waiting, then reports a confusing timeout; a partial file might be left behind.

What our system does: The client tries each configured server with a short connection timeout. When none answers, it prints `[CLIENT] No backup server reachable`, exits with an error in about 20 seconds (not 60), and creates no partial file. The dashboard shows that exact reason.

What changed technically (evidence for the teacher)

Changed files since the D1 update:

File	What it does	Edge cases it touches
`client/client.py`	The sender: chunks files, sends over TCP, reports resume points, detects source changes	1, 2, 5, 6, 8
`server/server.py`	The receiver: admits transfers, writes to `.part`, enforces quota, promotes only when complete	1, 3, 4, 5, 7
`common/protocol.py`	Message format between client and server (added the `MSG_ERROR` frame type)	4, 5
`mininet/bridge.py`	The link between dashboard and client:	1, 2, 4, 5, 8

File	What it does	Edge cases it touches
	retry, failover, disk-space checks, cancel	
`frontend/app.py`	Dashboard backend: start-transfer routing, verification, partial-file listing, cancel	1, 2, 3, 4, 5, 7, 8
`frontend/static/dashboard.js`	Dashboard screen logic: verification status, resume prompt	2, 7
`frontend/static/style.css`	Screen layout: grouped Transfer buttons, storage warnings	7 (and usability rework)
`frontend/templates/dashboard.html`	Dashboard page: Transfer menu reorganized into labeled rows	— (usability fix)
`mininet/serverctl.py`	Server start/stop and quota input	4
`tests/test_edge_cases.py`	New regression tests covering all eight cases	1–8
`docs/e2e-edge-cases.md`	Full register of every failure mode scanned	1–8

Key code locations to point at:

- `client/client.py:41` — tries each server with a short connect timeout; prints `[CLIENT] No backup server reachable` and exits 1 with no partial file.
- `client/client.py:72` — snapshots source size and modification time before the first chunk.
- `client/client.py:153` — catches `OSError` when the source is deleted mid-transfer and reports `[RESUME_POINT]` instead of crashing.
- `client/client.py:206` — reports the resume point before exiting after a server failure.
- `server/server.py:136` — rejects `.part` source names (`code=invalid`).

- `server/server.py:143` — rejects trailing dots/spaces in file names.
- `server/server.py:251` — write-time `OSError` becomes `MSG_ERROR`, so the client fails fast as a storage problem, not a network timeout.
- `server/server.py:289` — `os.replace` promotes `.part` to the final name only after the last chunk; the old copy survives until then.
- `mininet/bridge.py:342` / `:356` — failover disk check reads real disk usage instead of a stale quota number.
- `mininet/bridge.py:489` — clamps `resume_from` past the total chunk count (prevents a phantom success).
- `frontend/app.py:382` / `:384` — after a "done" result, verifies the stored file on both servers and sets `verified` / `verification_error`.
- `frontend/app.py:561` — auto-routes to the other server when the target is full; refuses when both are full.

Tests: about 18 new regression tests were added (the suite grew from roughly 72 to roughly 90), concentrated in `tests/test_edge_cases.py`, with related cases in `test_recovery.py`, `test_bridge_retry.py`, `test_serverctl.py`, `test_dashboard_server.py`, and `test_e2e_local.py`. Backend tests run without Mininet using fake sockets and temporary directories.

Quick demo script (what to show the teacher)

1. **Server dies mid-transfer** — start a large file transfer, kill the server (`h3 pkill -f server.py`), watch the dashboard show retry/resume, restart the server, confirm the file completes and its hash matches the source.
2. **Failed re-backup keeps the old copy** — back up a file, start a second transfer of the same name and stop it halfway; the original file stays byte-identical and only a `.part` remains; resume replaces it cleanly.
3. **Both servers unreachable** — stop both servers, start a transfer, confirm the client reports "No backup server reachable" in about 20 seconds with no partial file left behind.
4. **Storage full** — fill one server's disk, confirm the transfer is blocked with a clear storage reason instead of a silent truncation.

Full register: `docs/e2e-edge-cases.md`. Test list per case: `docs/test-cases.md`. Runbook: `docs/runbook/MASTER.md`.